

Team Knowledge Base

Everything about UrbanLens, in detail: architecture, every module with real constants, model training, judge Q&A; and code map · read once, know the system

0 · How to use this document

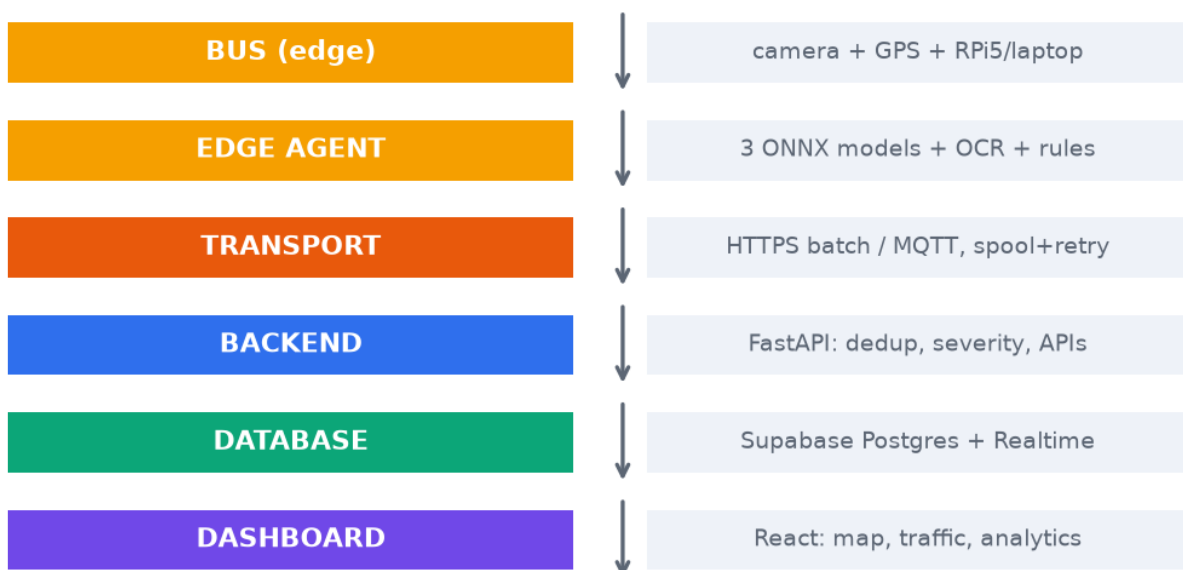
This is the team's single source of truth. Everything here is read directly from the actual code — every constant, formula and threshold quoted is real and lives in the file named next to it. Read it once fully, then use the Q&A; section (§9) before facing judges. If you can explain every row in the code map (§11) out loud, you know the system.

1 · What we are building — in plain words

UrbanLens turns city buses into a mobile sensing grid. A bus already drives every street of a city every day — we put a Rs.22,000 computer + camera + GPS on it, run AI on the video as it records, and send only the *conclusions* (not the video) to a cloud platform. Cities get a live map of potholes, waterlogging, garbage, congestion and traffic violations — without buying Rs.5 lakh CCTV poles that only see junctions.

- **Defects:** 9 classes of road problems detected and geo-tagged (pothole, crack, waterlogging, garbage_dump, illegal_parking, broken_streetlight, open_manhole, roadside_debris, faded_signage).
- **Traffic:** vehicles + pedestrians counted every 10 seconds; congestion index computed per road segment.
- **ANPR:** number plates read with confidence, GPS + timestamp; watchlist hits alert instantly.
- **Rash driving:** speeders relative to fleet + lane-weavers flagged with evidence.
- **Command centre:** multi-city dashboard — live map, congestion heat, analytics, incident workflow, alerts — updated in 1–3 seconds.

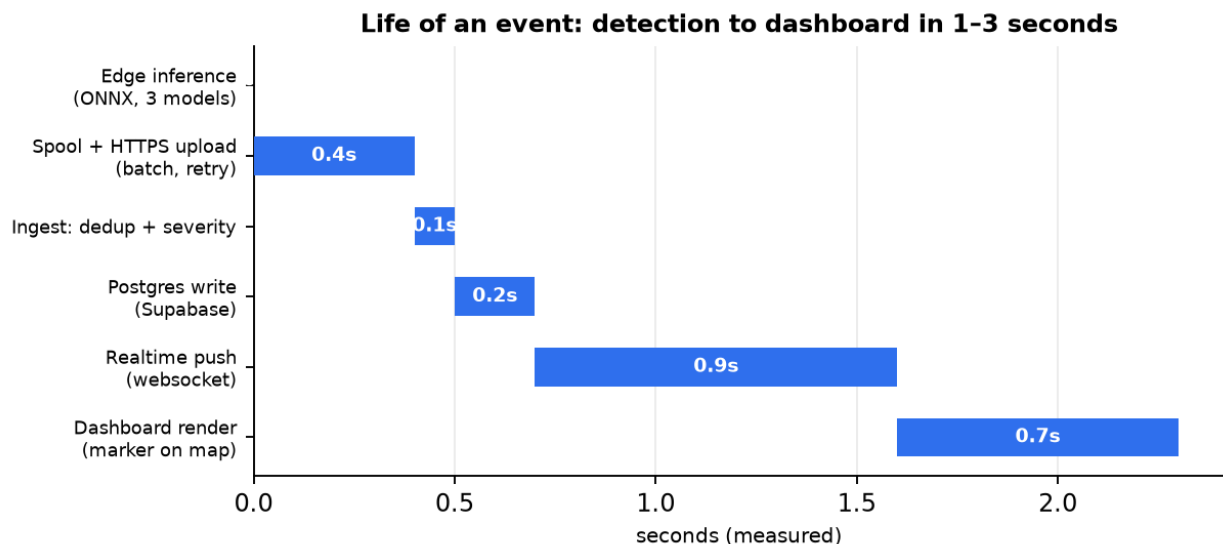
UrbanLens in one picture — six layers, raw video never leaves the bus



2 · Life of an event — a pothole, from camera to map

Memorise this story. It is the single most-asked demo question and touches every layer. Files are named so you can show the code.

1. **Camera frame** arrives (edge/camera.py — webcam, MJPEG stream or recorded video for demos).
2. **GPS position** is read (edge/gps.py — serial NMEA device, or route replay for demos; the agent refuses to tag an event without a fix).
3. **ONNX inference** (edge/detector.py): frame is resized to 640x640, run through the defect model in ~420 ms on CPU. Output is bounding boxes + class + confidence.
4. **Quality gates** (edge/main.py, _passes_gates): a detection is only believed if it passes confidence + streak + cooldown rules (details in §4). This is what kills false alarms.
5. **Privacy filter** (edge/privacy.py): a small JPEG crop around the pothole is created; any pedestrians inside are pixelated first. The raw frame is discarded — video never leaves the bus.
6. **Spool** (edge/buffer.py): the event is written to a local queue file first — so if the internet is down, nothing is lost.
7. **Upload** (edge/uploader.py): a background thread POSTs batches to the backend with retries (30 s timeout, 2 attempts).
8. **Ingest** (backend/app/ingest.py): the backend checks dedup — is there already an open pothole within 35 m in the last 240 minutes? (backend/app/dedup.py). If yes: observation_count++. If no: a new incident is created.
9. **Severity** (backend/app/severity.py): scored 1–10 from label risk x confidence x road priority x corroboration.
10. **Postgres write** (Supabase). Realtime publication pushes the change.
11. **Dashboard** (dashboard/src/...): the websocket fires; a React state update adds the marker. **1–3 s after the camera saw the pothole.**



3 · The edge agent — module by module

Entry point: edge/main.py. One loop per frame: read frame → read GPS → run 3 pipelines → gates → spool → upload. ~1.6 s/frame on a laptop CPU, same code targets a Raspberry Pi 5 (no GPU anywhere).

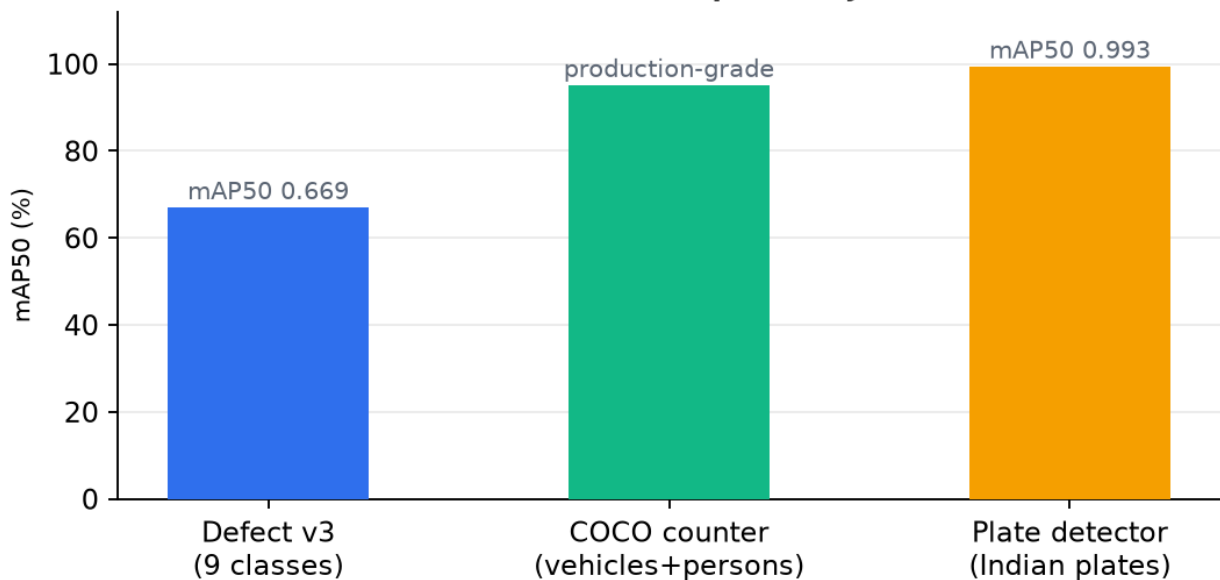
Module	What it does — with the real numbers
--------	--------------------------------------

camera.py	Uniform source for webcam (index), HTTP MJPEG stream, or video file for demos. Emits (frame_id, image) tuples.
gps.py	Serial NMEA GPS reader (real hardware) OR route replay: interpolates position along routes/route_XX.json by elapsed time, including a plausible speed (28 km/h on demo routes). Events are never geo-tagged without a fix.
detector.py	Defect model ONNX runner. Input 640x640; ultralytics export format output (1, 4+nc, N) = cx,cy,w,h,score per box → converted to corner coords, greedy NMS applied. Runs coco.onnx and plate.onnx too (same runner, different weights).
main.py gates	THE noise killer — a detection must pass: (1) per-class minimum confidence (garbage_dump 0.78, pothole 0.60, default 0.55), (2) a streak — seen at least 2 times within 1.5 s (motion-tolerant: counts sightings in time, not fixed position), OR a single hit at >= 0.75 confidence, (3) per-label cooldown 4 s (same label not re-reported within 4 s). Result: leaves/shadows/reflections never become incidents.
traffic_counter.py	COCO model counts vehicles (car, truck, bus, motorcycle, bicycle) + persons every frame; every 10 s (WINDOW_S) it flushes a window record: counts, avg GPS pos/speed, congestion index = min(100, vehicles x 12 + speed_penalty + crowd bump). Speed penalty = (1 - speed/40 km/h free-flow) x 40. Crowd bump +20 if >= 2 persons. Empty windows skipped. POSTs to /api/v1/traffic/segments.
anpr.py	Plate pipeline (see §5 for detail): plate.onnx detection → crop → RapidOCR → text cleaning against Indian plate regex $^{[A-Z]\{2\}\d{1,2}[A-Z]\{0,3\}\d{4}\$}$ → multi-frame voting (3 agreeing reads within 12 s, same track) → alert with plate+confidence+GPS, then 120 s cooldown per plate.

rash_detect or.py	Rule engine, no ML: greedy IoU tracker builds tracks (min 5 observations); flags (a) speeding — avg pixel speed > 2.0x fleet median, (b) lane weaving — >= 3 horizontal direction flips. Alert carries the evidence (speeds, flip count) and attaches the plate if ANPR saw one. 3/3 unit tests (edge/test_rash.py).
privacy.py	crop_evidence(): small JPEG (quality 70) with the defect circled + captioned. blur_boxes parameter pixelates pedestrian boxes (downscale to 8-px cells, upscale nearest) inside the crop before encoding — bystanders never leave the bus identifiable. Raw video is never uploaded, period.
buffer.py	Spool-to-disk store-and-forward. Upload failures re-queue; detection never blocks on the network.
uploader.py	HTTPS batch POST (token auth, 30 s timeout, retry) with an MQTT mode for brokered municipal networks.

4 · The three models — and how they were actually trained

The three models — all trained/exported by us, all run on CPU



Model	Training data	Result	Where the weights live

Defect detector v3	RDD2022 + Kaggle road-damage sets, merged and re-split with a 2x harder validation set; 9 classes	mAP50 0.669 (v1 was 0.580); open_manhole 0.971, pothole strong; iterative v1->v2->v3 with dataset fixes + close-mosaic aug	ml/train_v3.py -> ml/runs/urban/ensemble.onnx
COCO counter	No training — ultralytics YOLOv8n COCO pretrained	Near-perfect on vehicles (standard benchmark classes)	exported to edge/models/coco.onnx
Plate detector	Kaggle 'indian-license-plates-with-labels' — 2,083 images, split 1819 train / 202 val	mAP50 0.993, mAP50-95 0.846, P 0.984, R 0.987 — trained 40 epochs, ~8 min on RTX 5060 (ml/train_plate.py)	edge/models/plate.onnx

- **Why YOLOv8n (nano)?** 'n' is the smallest variant — designed for CPU/edge. Bigger variants gain a little accuracy but need GPUs we won't have on a bus.
- **Why ONNX?** PyTorch (the training format) is heavy to install on an edge device; ONNX Runtime runs the same weights as a portable 12–45 MB file with CPU optimisation. Training happens on the GPU laptop; deployment ships the ONNX file only.
- **Why is defect mAP 0.669 'good'?** 9 visually similar classes (crack vs pothole vs debris) on real messy street footage is one of the harder detection problems; published road-damage work reports similar ranges. open_manhole at 0.971 shows the model is excellent where it matters most (life-threatening hazards).
- **Model iteration is a story worth telling:** v1 0.580 → identified label-noise and easy validation split → v3 0.669 on a deliberately harder split. Judges like honest iteration more than a single suspicious number.

5 · ANPR pipeline — read a plate, step by step

- 1. plate.onnx** finds plate-shaped boxes (mAP50 0.993 — trained on Indian plates specifically, because international datasets miss our plate styles).
- 2. Crop + OCR:** RapidOCR (onnxruntime-based, fully offline, free) reads the crop. OCR fragments are x-sorted and joined; junk tokens (IND, stars) filtered.
- 3. Clean:** candidate text must match the Indian plate regex `^[A-Z]{2}\d{1,2}[A-Z]{0,3}\d{4}$` (state code, series, 4 digits) — anything else is discarded.
- 4. Track key:** plate box position quantised to an 80-px grid ($x//80$, $y//80$) — same vehicle across frames lands on the same key without needing a real tracker.
- 5. Voting:** 3 agreeing reads within a 12 s window are required before an alert. One bad OCR read can never become an alert.
- 6. Cooldown:** same plate not re-alerted for 120 s.
- 7. Alert:** POST `/api/v1/traffic/anpr` — plate, confidence, GPS, timestamp, kind (anpr/stolen/rash_driving). Watchlist matches fire as 'stolen'.

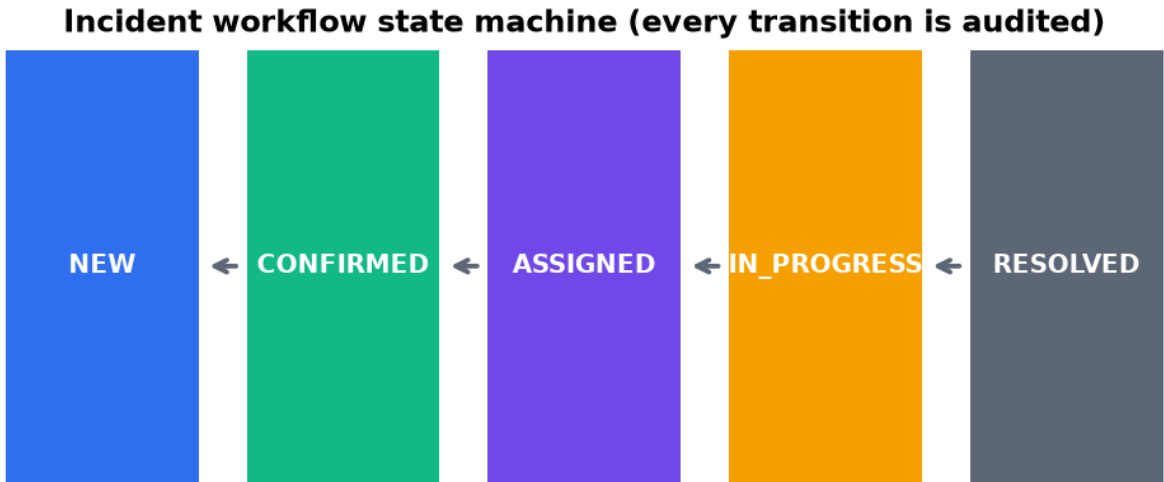
Verified on real video: KL01BM5673, GJ03LK0563, TN11AK9149, KA41EM7969 — read with 93–96% confidence and stored in the DB with full evidence. Honest limitation: very distant/angled plates are skipped by design (voting cannot confirm them).

6 · The backend — every endpoint, what it really does

FastAPI (Python), app factory in backend/app/main.py. Token-gated ingest (X-API-Token), OpenAPI docs auto-generated at /docs.

Endpoint	What it does
POST <code>/api/v1/ingest</code>	Observation intake: dedup lookup (35 m radius, 240 min window, same label — haversine distance after a bbox pre-filter) → increment existing incident or create new; severity scored on every update.
GET <code>/api/v1/incidents</code>	List/filter incidents; workflow transitions via state machine (NEW→CONFIRMED→ASSIGNED→IN_PROGRESS→RESOLVED) with audit rows; each incident carries evidence crop, severity 1–10, department routing.
GET <code>/api/v1/traffic/segments</code>	10-second counting windows per bus — the congestion time series.
GET <code>/api/v1/traffic/bottlenecks</code>	Aggregates segments into ranked bottleneck corridors (by congestion x duration).
GET <code>/api/v1/traffic/od + /delays</code>	Origin-destination flow patterns and route delay estimation — planner-grade analytics from free bus data.
POST <code>/api/v1/traffic/anpr</code>	ANPR / rash / stolen alerts with plate, confidence, GPS, notes.

GET /api/v1/traffic/alerts	Alert feed for the dashboard.
GET /api/v1/weather, /aqi	Open-Meteo + OpenAQ overlays (free APIs, Rs.0).
GET /api/v1/routing	Smart routing: avoids defect-dense + congested segments.
WebSocket / realtime	Supabase Realtime publication on 4 tables → dashboard websockets (see §7).



Severity formula (backend/app/severity.py) — learn it: $\text{severity} = \text{base_risk}(\text{pothole } 7, \text{ open_manhole } 9, \text{ crack } 4 \dots) \times \text{confidence_factor}(0.7\text{--}1.1) \times \text{corroboration}(1.0\text{--}1.5, \text{ capped}) \times \text{road_priority}/5 \rightarrow \text{clamped } 1\text{--}10$. Meaning: a high-confidence open manhole on a major road that 4 buses saw = 10.

The reliability war story (know this cold): on Supabase's free tier, burst uploads exhausted the session pooler (EMAXCONNSESSION, 15-client cap) — uploads hung and timed out. Fix: switch to the transaction-mode pooler (port 6543), disable prepared statements (prepare_threshold=None — pgbouncer can't cache them), and use NullPool in SQLAlchemy so no stale connections are reused. Result: 40/40 burst uploads succeed. This is a real production lesson, not a hackathon patch.

7 · The dashboard — every view and the realtime trick

React 18 + Vite + Leaflet (CARTO basemaps). Two data paths: REST for bulk loads, Supabase Realtime websockets for push. The browser holds only an anon key (read-only) — the service key never ships to the client.

View	What it shows
Live Map	All incidents across cities — severity colour, photo evidence, status; realtime markers appear in 1–3 s.
Traffic	Congestion heat + per-segment vehicle counts, speed, persons; bottleneck ranking.
Incidents	Workflow board — confirm/assign/resolve with audit trail; department routing.
Analytics	Hourly congestion patterns, defect class distribution, city comparisons.
Media	Evidence crops uploaded from the fleet.

Live Cam / Fleet / Smart Route	Camera feed preview; per-bus strip (vehicle code, type, speed); route planner avoiding defects/congestion.
--------------------------------	--

Realtime detail worth knowing: React 18 StrictMode mounts every component twice in dev — naive websocket code double-subscribes and leaks channels. Ours uses per-call channel ownership + a refreshRef pattern so remounts clean up correctly. There is also a polling fallback if websockets drop — the dashboard never goes blind.

8 · Running it — the exact commands

- **Cloud mode (normal):** scripts/start_backend.bat (FastAPI → Supabase Postgres) · npm run dev in dashboard/ → http://127.0.0.1:5173 · scripts/run_edge.bat (agent: --vehicle DL01CJ8943 --type bus --video media/route_demo.mp4 --route routes/route_01.json).
- **Offline mode:** scripts/start_backend_sqlite.bat — same backend on the local urbanlens.sqlite3 file; everything works except websocket push (dashboard falls back to polling). Useful when the network blocks Supabase ports (happens on some college/office WiFi).
- **Public demo URLs:** Cloudflare quick tunnels expose backend + dashboard over HTTPS (URLs rotate on restart).
- **Hardware path:** identical agent code runs on Raspberry Pi 5 + camera + GPS + 4G dongle ≈ Rs.22,000 per bus (BOM: Pi 5, camera module, GPS module, 4G dongle, housing, install).

9 · Judge Q&A; — learn these answers

A. Computer-vision fundamentals (the 'do you actually know ML' probes)

Q	A — say it like this
What is mAP50?	Mean Average Precision at IoU threshold 0.5: for each class, precision-recall is computed at 0.5 overlap between predicted and ground-truth boxes, averaged over classes. 0.669 means 66.9% — strong for 9 confusable classes on real street video.
What is IoU?	Intersection over Union — the overlap area between predicted and ground-truth boxes divided by their union. The standard match criterion for detection.
What is NMS?	Non-Maximum Suppression: YOLO emits many overlapping boxes for one object; NMS keeps the highest-confidence box and suppresses overlaps above a threshold. We run greedy NMS in edge/detector.py.
Why ONNX at the edge?	Training uses PyTorch on a GPU; deployment ships a portable ONNX file that runs on CPU via ONNX Runtime — no heavy framework on the bus computer, hardware-portable (laptop today, Pi 5 tomorrow).
Why not fine-tune a big model?	nano variants are built for CPU edge; larger models trade latency/power for marginal accuracy we don't need for triage (a human confirms before a work order anyway).
Why is 0.669 acceptable?	Triage, not diagnosis: our gates + dedup + corroboration mean a missed frame is caught by the next bus; a false positive costs one verification click. Safety-critical classes score highest (open_manhole 0.971).

How do you handle night/rain?	Quality gates suppress low-confidence frames rather than emitting noise; degraded conditions = fewer reports, not wrong reports. Night-augmented retraining is a data-collection step away — the pipeline exists.
-------------------------------	---

B. System & data (the 'is this real engineering' probes)

Q	A
What happens when the internet drops?	Detection continues; events spool to disk (edge/buffer.py) and upload on reconnect with retries. Proven: during a real Supabase outage the agent queued 21 records and flushed them on recovery — detected and uploaded throughout.
Why Postgres/Supabase?	Relational fits our workflow + analytics (joins, group-bys); Supabase adds realtime websockets and a free tier. We hit the pooler limits and fixed them properly (transaction pooler + NullPool) — 40/40 burst uploads.
40 buses see one pothole — 40 incidents?	No: haversine dedup within 35 m / 240 min / same label merges them into one incident with observation_count — that count feeds severity and confidence.
How is congestion actually computed?	Per 10-second window: $\min(100, \text{vehicles} \times 12 + (1 - \text{speed}/40) \times 40 [+ 20 \text{ if } \geq 2 \text{ persons}])$. Counts + speed deficit + crowd, capped at 100. Simple, explainable, tunable.
Scale to 1,000 buses?	Ingest is stateless + pooled DB → horizontal. Real costs appear at DB scale (~\$25/mo managed Postgres) and evidence storage. MQTT per depot is designed in for broker mode.
Why not stream video to the cloud and run AI there?	Bandwidth (a 4G dongle cannot stream 30 fps from a fleet), privacy (faces/plates would transit networks), and cost. Edge inference sends conclusions — kilobytes, not gigabytes.

C. Privacy, safety, and product (the 'should this exist' probes)

Q	A
You read plates — privacy?	Raw video never leaves the bus. Evidence crops pixelate bystanders before encoding. ANPR stores plate text + confidence + GPS only — no face imagery. We position alerts as intelligence for authorities with due process, not automated punishment. Aligns with DPDP-Act principles: purpose limitation + data minimisation.
False rash-driving accusations?	Alerts are evidence-based (measured speeds, lane-change counts, 5-frame minimum track) with human confirmation before action; per-vehicle cooldowns prevent spam. We calibrated thresholds conservatively (2x fleet median).
What is the weakest part?	Ground-speed calibration for rash driving — pixel-space heuristics flag reliably but court-grade speeds need GPS-correlated calibration. Say it confidently; honesty scores.
Why will a transport corporation cooperate?	Zero capex, zero driver workload, revenue share on data products. The bus is already driving the route.
What did it all cost?	Rs.0 in external APIs — Supabase free tier, Open-Meteo, OpenAQ, CARTO basemaps, Cloudflare tunnels. Training was free (public datasets + own GPU). Prototype hardware ≈ Rs.22k/bus.
What is the competitive edge?	Cost per sensing point (Rs.22k vs Rs.1–5L) AND coverage — buses sweep interior streets that fixed CCTV never sees; complaint apps depend on citizens doing the sensing.

10 · The numbers to quote (all measured)

Number	Context
0.669 / 0.971	defect model mAP50 / open_manhole mAP50 (v3, hard split)
0.993	plate detector mAP50 (2,083 Indian plates, 40 epochs)
19/19	vehicles counted on demo route = exact ground truth
93–96%	ANPR read confidence on real plates (KL01BM5673 ...)
1–3 s	detection → dashboard latency (websockets)
40/40	burst upload success after pooler fix
83/83	automated dashboard UI checks (Puppeteer)
Rs.0	external API spend — entirely free tiers
Rs.22,000	per-bus hardware BOM (Pi 5 + camera + GPS + 4G)
~1.6 s/frame	all three models, CPU only, no GPU

11 · Code map — say what every file does

File	One-line answer
edge/main.py	Agent loop: frame → GPS → 3 pipelines → gates → spool → upload; all thresholds live here
edge/detector.py	ONNX runner + NMS + coordinate conversion for all 3 models
edge/traffic_counter.py	10-second counting windows → congestion index
edge/anpr.py	Plate detect → OCR → regex clean → 3-frame vote → alert
edge/rash_detector.py	IoU tracking + speed/weaving heuristics → evidence alerts
edge/privacy.py	Evidence crop with circle highlight + pedestrian pixelation
edge/buffer.py, uploader.py	Offline spool; HTTPS/MQTT upload with retry
backend/app/ingest.py, dedup.py	Observation intake; 35 m / 240 min same-label merging
backend/app/severity.py	1–10 severity: base risk x confidence x corroboration x road priority
backend/app/incidents.py	Incident CRUD + workflow state machine + audit
backend/app/traffic.py	Segments, bottlenecks, OD, delays, ANPR alerts
backend/app/database.py	Engine per mode: SQLite / Supabase tx-pooler (prepare_threshold=None + NullPool)
dashboard/src/lib/realtime.js	StrictMode-safe websocket channels + polling fallback
dashboard/src/components/*	LiveMap, TrafficView, IncidentFeed/Detail, Analytics, Fleet, SmartRoute, LiveCam, Uploads
ml/train_v3.py, train_plate.py	Defect v3 training; plate detector training (GPU, exported to ONNX)

12 · Glossary (30-second versions)

- **YOLOv8**: one-shot object detector — single network pass finds all boxes+classes, fast enough for CPU.
- **ONNX**: open neural-network exchange format; ONNX Runtime executes it without PyTorch.
- **NMS**: removing duplicate overlapping detections, keeping the most confident.
- **IoU**: box-overlap metric; also the basis of our vehicle tracker.
- **mAP50**: detection accuracy averaged over classes at 0.5 IoU matching.
- **Spool**: local disk queue so uploads survive network loss.
- **Dedup**: merging repeated sightings of the same physical defect into one incident.
- **Transaction pooler**: Supabase's connection router (port 6543) — fixed our free-tier connection limits.
- **Congestion index**: 0–100 composite of vehicle count, speed deficit vs 40 km/h free-flow, and crowd presence.
- **StrictMode**: React dev-mode double-mounting — why our websocket code cleans up channels explicitly.